Training Leaves Traces: Diagonal-Dominant Fingerprints for White-Box Model Lineage Verification

Anonymous Submission

Abstract

Open-weight models are frequently reused through fine-tuning, quantization, pruning, and merging, but their ancestry is rarely documented in a reliable way. Hashes stop working once weights change, metadata may be incomplete or misleading, and watermarks help only when they are inserted before release. We therefore ask a retrospective question: **given two checkpoints with compatible residual architectures, can we determine whether they share weight ancestry?**

We find that training leaves a measurable trace in residual blocks. When a block's input and output projections are paired correctly, their product develops a diagonal-dominant structure. This pattern is absent at initialization and does not emerge in matched networks without skip connections. We use the structure to recover block correspondences and then compare centered residual signatures to obtain a model-level lineage score. Across 9 model families spanning language, vision, and speech - including GPT-2, LLaMA-2, ViT, Whisper, and ResNets - the method recovers the correct projection pairings with 80–100% accuracy. In our lineage benchmarks, checkpoints produced through fine-tuning, pruning, quantization, and LoRA merging remain close to their ancestors, whereas independently trained models and distilled students do not. This distinction matters because a distilled student may reproduce a teacher's behavior without inheriting its weights. The resulting method provides a passive, post-hoc provenance signal for white-box residual models without relying on metadata, watermarks, training data, or forward passes.

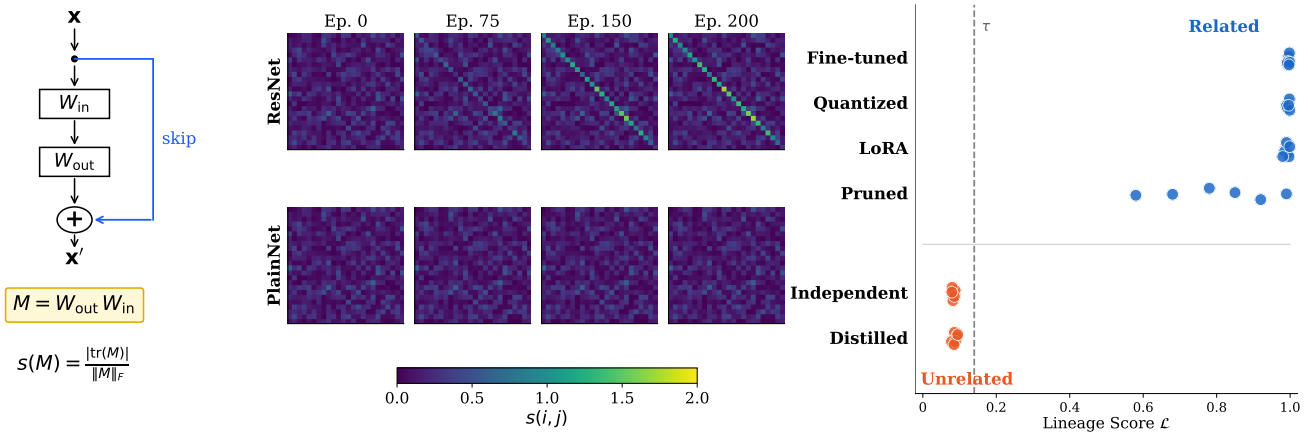
1 Introduction

The open-weight ecosystem has transformed how neural networks are distributed and reused. When Meta released LLaMA (Touvron et al. 2023), the community produced instruction-tuned variants within weeks. Platforms like Hugging Face now host hundreds of thousands of model checkpoints (Hugging Face 2024), many of which are fine-tuned (Devlin et al. 2019; Ouyang et al. 2022), quantized (Han, Mao, and Dally 2016; Dettmers et al. 2022), pruned (Han et al. 2015), or merged (Wortsman et al. 2022; Ilharco et al. 2023). This creates a *model supply chain* where downstream checkpoints inherit weights from upstream models, but the lineage is often undocumented or intentionally obscured. Provenance of AI-generated content is now treated as a first-class problem: Google's SynthID watermarks text and images at generation time (Dathathri et al. 2024), yet no analogous mechanism exists for the models themselves. Model reuse is easy; reliable model provenance is not.

Existing provenance mechanisms fall short (Table 1). Cryptographic hashes break after any weight modification. Metadata and model cards (Mitchell et al. 2019) require honest distributors and preserved records; they can be falsified or lost. Watermarks (Uchida et al. 2017; Adi et al. 2018) must be inserted before release; they cannot be applied retroactively to already-deployed checkpoints, and no surveyed scheme proved robust against fine-tuning or pruning (Lukas et al. 2022). Behavioral tests that compare model outputs cannot distinguish weight inheritance from imitation: two models can behave similarly without sharing weights, especially after distillation (Hinton, Vinyals, and Dean 2015). Activation-based similarity measures like CKA (Kornblith et al. 2019) and decision-boundary fingerprints like IPGuard (Cao, Jia, and Gong 2021) inherit this limitation. We need a passive, post-hoc signal that reads provenance from the weights themselves.

We ask: given a reference checkpoint and a suspect checkpoint with the same residual architecture (e.g., both LLaMA-2-7B derivatives), can we determine whether the suspect inherited weights from the reference, even after common transformations like fine-tuning, quantization, or pruning? This is distinct from duplicate detection (which hashes solve until the first modification), behavioral similarity (which conflates imitation with inheritance), and watermark verification (which requires forethought). The question is whether shared weight ancestry leaves detectable structure after modification.

The answer is yes. We find that training itself leaves a characteristic fingerprint in the weights of residual networks. In these architectures (He et al. 2016), each block adds a learned transformation to its input via a skip connection (Figure 1a). During training, the input and output projections of each block align so that their product $W_{\text{out}}W_{\text{in}}$ develops diagonal-dominant structure that is absent at random initialization and does not appear in networks without skip connections (Figure 1b). This structure is a product of gradient coupling: because W_{in} and W_{out} sit in the same residual branch, every backward pass sends correlated updates through both, and the correlation accumulates into diagonal structure in their product. The fingerprint is absent at initialization, never appears without the skip connection, and can be destroyed or created by manipulating the coupling alone (Section 5.1).



(a) Residual Branch Product **(b) Training Induces Diagonal Dominance: ResNet vs PlainNet** **(c) White-Box Model Lineage Verification**

Figure 1: Training leaves traces in residual networks. (a) The residual branch product $M = W_{\text{out}}W_{\text{in}}$ for a block with skip connection; the diagonal-dominance ratio $s(M)$ measures trace concentration. (b) Training dynamics: score matrices $s(i, j)$ from epoch 0 to 200. ResNet (top row) develops strong diagonal structure; PlainNet without skip connections (bottom row) remains random. The fingerprint is learned, not an initialization artifact. (c) Lineage verification: the lineage score $\mathcal{L}$ (Eq. 7) aggregates per-block signature similarity. Related checkpoints (fine-tuned, quantized, pruned, LoRA-merged) score high; unrelated checkpoints (independent, distilled) score low, enabling clean separation.

We exploit this fingerprint for provenance verification. Each residual block yields a signature: its branch product scored for diagonal concentration via a trace/Frobenius ratio (Equation 3). Correctly paired blocks from the same trained model score high, while mismatched or unrelated blocks do not. The extraction is architecture-aware, respecting the specific structure of Transformer MLPs, attention paths, and ResNet bottleneck blocks (Table 2). To compare two checkpoints, we align their blocks using these signatures, aggregate the pairwise similarities into a model-level lineage score, and threshold it against a null distribution built from unrelated models (Equation 7).

The fingerprint is not specific to one model family. We observe it in public checkpoints across language, vision, and speech (GPT-2, BERT, LLaMA-2, Mistral, Qwen2.5, DeepSeek-R1, ViT, Whisper, and ResNets) where it recovers block correspondences with 80–100% accuracy (Table 3). Turning to the verification task itself, we construct controlled lineage benchmarks on MLPs and ResNet-18: models with genuine weight ancestry (produced by fine-tuning, weight noise, pruning, quantization, or LoRA merging) are cleanly separated from those without (independent training runs and distilled students), with AUROC = 1.000. The distillation case is instructive: a student can closely replicate its teacher's outputs, yet our score correctly labels it unrelated, because its weights carry no trace of the teacher's training (Figure 1c). Finally, on the public LLaMA-2 family, the method flags three real fine-tuned descendants (chat, Vicuna, CodeLlama) as related while rejecting OpenLLaMA-7B, which shares the architecture but inherits no weights.

Our contributions:

- **We uncover a training-induced structure in residual-network weights.** Correctly paired projections develop diagonal-dominant products during training; the pattern is absent at initialization and in skip-free controls.
- **We turn this structure into a white-box lineage test.** We factor out the diagonal component common to all trained models and compare the residual signatures to obtain block correspondences and a lineage score, requiring no data or watermarks.
- **We test across architectures and real model families.** Block correspondences reach 80–100% accuracy on 9 model families; the lineage score separates inherited from independent checkpoints on controlled & public benchmarks.
- **We show the signal survives substantial modification.** It persists through fine-tuning, quantization, pruning, LoRA merging, and RLHF, and tracks partial inheritance in layer grafts and model merges.

2 Problem Setting and Related Work

2.1 White-Box Model Lineage Verification

A verifier receives two checkpoints A and B and determines whether they share weight-level lineage. We assume white-box access and compatible residual architectures (same L and d); cross-architecture comparisons are out of scope. The verifier operates on weights alone: no training data, activations, or forward passes.

Checkpoints are *related* if they share a common weight ancestor through fine-tuning, RLHF, pruning, quantization, or LoRA merging. They are *unrelated* if initialized independently, even when architecture, data, task, or outputs match; a distilled student trained from scratch is unrelated to its teacher, even if it mimics the outputs perfectly. Linear merges induce partial lineage (Appendix D.1). The verifier returns RELATED or UNRELATED. The score is symmetric, $\mathcal{L}(A, B) = \mathcal{L}(B, A)$; directional attribution requires meta-

Method	Retro.	FT	Quant	Distill
Crypto Hash	✓	✗	✗	✗
Metadata/Logs	✓	✓	✓	✓
Watermarking	✗	∼	∼	✗
Behavioral Fingerprints	✓	✓	✓	✗
Ours	✓	✓	✓	✓

Table 1: Comparison with prior provenance methods. Retro.: applicable retroactively. FT: survives fine-tuning. Quant: survives quantization. Distill: correctly classifies distilled copies as unrelated. ∼: partial or unreliable.

Architecture	Residual branch $F(x)$	Product M
Transformer MLP / BasicBlock	$W_2\sigma(W_1x)$	W_2W_1
Attention V/O path	$W_O\mathrm{Attn}(x)W_Vx$	W_OW_V
Attention Q/K path	bilinear $x^\top W_QW_K^\top x$	$W_QW_K^\top$
Bottleneck ResNet	$W_3\sigma(W_2\sigma(W_1x))$	$W_3W_2W_1$
SwiGLU MLP	$W_{\mathrm{down}}[\sigma(W_{\mathrm{gate}}x)\odot W_{\mathrm{up}}x]$	$W_{\mathrm{down}}W_{\mathrm{up}}$

Table 2: Architecture-aware residual branch products. Each M composes the linear maps of one branch (dropping nonlinearities) so it maps the residual stream to itself. Factorization must match the architecture; incomplete products destroy the signal (Appendix D.2).

data (Appendix B). The method complements cryptographic signing and watermarking where those were never applied.

2.2 Related Work

Table 1 summarizes the comparison.

Cryptographic hashes break after any weight modification. Model cards (Mitchell et al. 2019) require honest distributors and can be falsified when checkpoints are redistributed. Proof-of-learning (Jia et al. 2021) requires retained training transcripts that already-released checkpoints lack.

Parameter watermarks (Uchida et al. 2017) and backdoor watermarks (Adi et al. 2018) embed deliberate signals during training. Lukas et al. (2022) found no surveyed scheme that reliably survived fine-tuning and pruning, and all require proactive insertion: unwatermarked legacy checkpoints cannot be verified.

IPGuard (Cao, Jia, and Gong 2021) extracts inputs near classification boundaries; adversarial frontier stitching (Le Merrer, Perez, and Trédan 2020) marks decision surfaces with adversarial examples. CKA (Kornblith et al. 2019) and SVCCA (Raghu et al. 2017) compare representations on a probe dataset. All four measure functional similarity and cannot distinguish inheritance from imitation: a distilled student may share decision surfaces while having independent weights. They also require data and forward passes.

Weight cosine, Frobenius distance, and permutation matching (Ainsworth, Hayase, and Srinivasa 2023) detect derived checkpoints when the suspect remains close in parameter space, but these are uncalibrated global scalars with no structural account of lineage. HuRef (Zeng et al. 2025) relies on LLM parameter convergence after pretraining. REEF (Zhang et al. 2025) needs probe data or activations. MoTHer (Horwitz, Shul, and Hoshen 2025) recovers model trees but requires a checkpoint collection rather than a single pair. Our fingerprint is data-free, operates on a single pair against a calibrated null, and localizes inheritance to specific blocks.

3 Training-Induced Diagonal Dominance in Residual Branch Products

A residual block computes $x_{\ell+1} = x_\ell + F_\ell(x_\ell)$, where F_ℓ is the residual branch. For a branch with K linear layers $W_1, \ldots, W_K$ interleaved with nonlinearities, the *residual branch product* composes the linear factors, dropping nonlinearities:

$$M_\ell = W_K W_{K-1} \cdots W_1 \in \mathbb{R}^{d\times d} \tag{1}$$

Individual factors W_k are rectangular and do not share an input–output basis; their composed product maps the residual stream back into its own coordinate space, so its diagonal entries compare like with like. The exact factorization is architecture-dependent (Table 2).

3.1 Why Residual Training Induces Diagonal Dominance

Under the Frobenius inner product $\langle A, B\rangle = \mathrm{tr}(A^\top B)$, the identity matrix and the traceless matrices span orthogonal subspaces of $\mathbb{R}^{d\times d}$. Any branch product therefore splits into a component along the identity, with coefficient $\langle M_\ell, I\rangle/\langle I, I\rangle = \mathrm{tr}(M_\ell)/d$, and an orthogonal traceless remainder:

$$M_\ell = -\varepsilon_\ell I + E_\ell \tag{2}$$

where $\varepsilon_\ell = -\mathrm{tr}(M_\ell)/d$ and $\mathrm{tr}(E_\ell) = 0$. Traces are predominantly negative (Appendix C.2), so ε_ℓ is typically positive; the score uses magnitude.

The *diagonal-dominance ratio* (Park 2026) measures this concentration:

$$s(M) = \frac{|\mathrm{tr}(M)|}{\|M\|_F}, \tag{3}$$

the fraction of the matrix's energy along the identity direction, up to $\sqrt{d}$. Training moves energy into the identity component: correctly paired branch products score far above unpaired or untrained ones (Figure 1b).

The skip connection changes what the branch must learn. A plain layer must transmit the entire representation; a residual branch learns only a correction around an identity path that already transmits everything. This has a gradient consequence: W_{in} and W_{out} are the two ends of one correction, and the loss reaches both through the same branch. ∇W_{out} depends on the branch input, a function of W_{in}; ∇W_{in} depends on the upstream gradient, which flows through W_{out}. Their updates are therefore correlated at every step by construction. In a plain network no such pairing is privileged, every adjacent pair of layers is coupled equally so no per-block signature can form.

The correlation accumulates along the identity direction because the skip means the branch learns a correction, not the full map. Small coordinated corrections to an identity baseline accumulate along the diagonal. Section 5.1 shows that injecting this component directly, without backpropagation, suffices to build the fingerprint.

A natural alternative account is dynamical isometry (Pennington, Schoenholz, and Ganguli 2017): if training enforced near-orthogonal block Jacobians $J = I + J_F$, diagonal dominance would follow as a byproduct of pressure toward $J \approx I$. This means trained Jacobians should be more orthogonal than at initialization, which we do not observe in Section 5.1.

3.2 Margin Analysis

By orthogonality of decomposition (2), $\|M_\ell\|_F^2 = \varepsilon_\ell^2 d + \|E_\ell\|_F^2$: the score depends on M only through the ratio of identity to traceless components:

Proposition 1 (Correct-pair score)**.** *Let $M = -\varepsilon I + E$ with $\mathrm{tr}(E) = 0$ and $\varepsilon \neq 0$. Then:*

$$s(M) = \frac{\sqrt{d}}{\sqrt{1 + \|E\|_F^2/(\varepsilon^2 d)}} \leq \sqrt{d} \tag{4}$$

with equality if and only if $E = 0$.

Proof. $|\mathrm{tr}(M)| = |\varepsilon| d$ and $\|M\|_F^2 = \varepsilon^2 d + \|E\|_F^2$ by orthogonality. Substitute into (3).

Geometrically, $s(M)/\sqrt{d}$ is the cosine of the angle between M and the identity. On the other hand, mismatched products have no such concentration: W_{out} and W_{in} from different blocks have no preferred alignment, giving

Corollary 1 (Mismatched baseline)**.** *For products of projections from independent blocks, $\mathbb{E}[s] \approx 1/\sqrt{d}$.*

The margin scales as d: wider networks separate correct from incorrect pairings more cleanly. Empirically the correct-pair score grows as $d^{0.87}$ (Section 5.2).

4 From Diagonal-Dominance Fingerprints to Lineage Verification

Diagonal dominance certifies trained residual structure, not ancestry: unrelated trained models both exhibit it. The decomposition (2) separates the two signals. The identity component is generic across trained models. The traceless component E_ℓ is checkpoint-specific: it survives weight-preserving transformations but not reinitialization. Verification runs on E_ℓ, with diagonal dominance gating which branches are trustworthy.

4.1 Centered Residual Signatures and Block Alignment

For each branch product M_ℓ we retain only the traceless remainder of decomposition (2), normalized to a unit vector:

$$R_\ell = M_\ell - \tfrac{\mathrm{tr}(M_\ell)}{d} \cdot I, \quad \phi_\ell = \tfrac{\mathrm{vec}(R_\ell)}{\|\mathrm{vec}(R_\ell)\|_2} \tag{5}$$

$R_\ell = E_\ell$ from (2). The signature ϕ_ℓ carries checkpoint-specific geometry that weight-preserving transformations inherit and independent training cannot reproduce. Because M_ℓ is invariant under hidden permutation and reciprocal rescaling, so is every signature built on it (Section 5.4).

Given reference A and suspect B with L blocks each, we form the similarity matrix $G_{ij} = \langle \phi_i^A, \phi_j^B \rangle$, gated by diagonal dominance (Appendix B), and recover the block correspondence via the Hungarian algorithm (Kuhn 1955):

$$\pi^* = \arg\max_{\pi \in \mathcal{P}_L} \sum_i G_{i,\pi(i)} \tag{6}$$

For derived checkpoints preserving block order, π^* should be the identity; recovering it without metadata confirms that signatures track block identity. Complexity: branch products $O(Ld^2h)$, similarity matrix $O(L^2d^2)$, assignment $O(L^3)$.

4.2 Lineage Score and Verification Test

Diagonal dominance certifies trained residual structure, not ancestry: a verifier built on the dominance score alone achieves AUROC=0.417 (Figure 6). To distinguish lineage, we compare the centered signatures ϕ_ℓ. The lineage score averages signature similarity over aligned branches:

$$\mathcal{L}(A, B) = \frac{1}{L} \sum_{\ell=1}^{L} \langle \phi_\ell^A, \phi_{\pi^*(\ell)}^B \rangle \tag{7}$$

Each term is a cosine, so $\mathcal{L} \in [0, 1]$ with $\mathcal{L}(A, A) = 1$. The score is symmetric. Related checkpoints score near 1; unrelated ones near 0, since independently initialized weights produce uncorrelated E_ℓ.

We calibrate against a null distribution $\mathcal{N}_A = \{\mathcal{L}(A, U_j)\}$ from independently trained models U_j, with mean μ_0 and standard deviation σ_0:

$$Z(A, B) = \frac{\mathcal{L}(A, B) - \mu_0}{\sigma_0} \tag{8}$$

The protocol returns

$$V(A, B) = \begin{cases} \textsc{Related} & \text{if } Z > \tau \\ \textsc{Unrelated} & \text{otherwise} \end{cases} \tag{9}$$

Threshold $\tau = 3.0$ gives false-positive rate below 0.13% under a Gaussian null (Appendix B). Compatibility (same L and d) is checked before scoring; the protocol returns INCOMPATIBLE otherwise. Memory: $O(L^2 + d^2)$ beyond the checkpoints.

5 Experiments

We validate the claims of Sections 3–4: fingerprint existence (Section 5.1), cross-architecture generalization (Section 5.2), lineage verification (Section 5.3), baseline comparisons (Section 5.4), and survival under post-training modification (Section 5.5).

5.1 The Fingerprint Exists and Is Learned

Section 3.1 predicts that the fingerprint is learned, requires the skip connection, and arises from gradient coupling rather than isometry. We test all three.

We train depth-24 residual MLPs (in_dim=24, hidden=64) on CIFAR-10 (Krizhevsky 2009) for 200 epochs

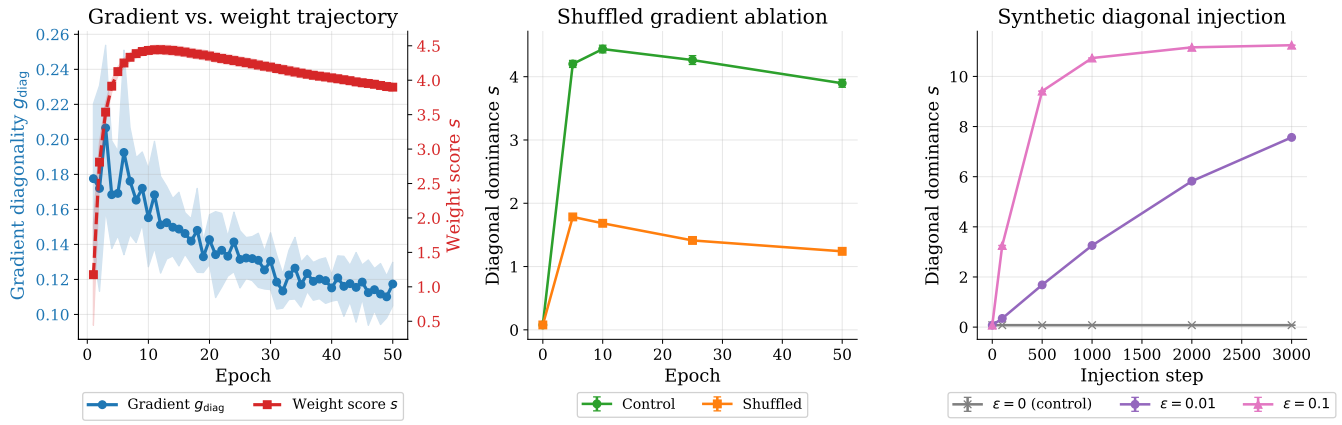


Figure 2: (a) Gradient diagonality g_{diag} (blue) stays flat at $\sim$0.15 while the weight score s (orange) rises to $\sim$4.0: individual gradient updates are not diagonal, but their accumulation is. (b) Shuffling ∇W_{out} across blocks (orange) drops the final score by 68% compared to control (blue), showing that within-block coupling is necessary. (c) Injecting synthetic diagonal updates $\Delta W = -\varepsilon \cdot e_i^\top$ directly into weights builds the fingerprint from scratch without any backpropagation, showing that coupling is sufficient.

(3 seeds) under seven initialization schemes: orthogonal, Kaiming-normal/uniform, Xavier-normal/uniform, uniform, and Gaussian σ=0.02. Input-output block pairing accuracy starts at chance ($\leq$2.1%) and reaches 93–100% after training for every scheme that converges (Appendix C.1); the chance-level baseline follows from a null model in which all pairs score uniformly (Appendix A). Orthogonal initialization satisfies dynamical isometry exactly at initialization yet yields zero correct pairs, confirming that the fingerprint requires training.

Using the same architecture, we compare ResNet-24 against PlainNet-24, which removes only the skip connection. ResNet-24 reaches 100% pair accuracy (AUROC 0.98); PlainNet-24, trained to similar loss (1.56 vs. 1.35), stays at chance (3%, AUROC 0.51). Training alone does not produce the fingerprint; the skip connection is required.

Then, we measure the normalized Jacobian deviation $\delta_J^{\text{norm}} = \|J^\top J/\|J\|_F^2 - I/d\|_F$ across the GPT-2 family (small/medium/large/XL), comparing pretrained weights against random initialization. Trained GPT-2 Jacobians are 5–12$\times$ less orthogonal than at initialization (GPT-2-small: 0.297 vs 0.025). The fingerprint grows while orthogonality decays, ruling out isometry (Appendix C.2).

To test whether gradient updates are themselves diagonal, we train 48-block residual MLPs (in_dim=128, hidden=256, CIFAR-10, 50 epochs, 3 seeds) and measure gradient diagonality $g_{\text{diag}} = |\text{tr}(\nabla W_{\text{out}} \nabla W_{\text{in}})|/\|\cdot\|_F$ alongside the weight score $s(M)$. The gradient product stays flat ($\sim$0.15), meaning individual updates are not diagonal. Yet the weight score climbs from 0.08 to $\sim$4.0 (Figure 2a). The diagonal structure emerges from many small correlated updates accumulating over training, not from any single diagonal gradient step.

Two interventions isolate gradient coupling as the cause. *Necessity:* shuffling ∇W_{out} across blocks before each optimizer step (block i's W_{out} receives gradients from a random block j) breaks only the within-block pairing while preserving valid gradients. This cuts the fingerprint by 68% (s: 3.90 $\rightarrow$ 1.24) at comparable accuracy (47% vs 54%) (Figure 2b). *Sufficiency:* injecting synthetic updates $\Delta W_{\text{in}} = -\varepsilon \cdot e_i$, $\Delta W_{\text{out}} = -\varepsilon \cdot e_i^\top$ (ε=0.01) directly into weights for 3000 steps, with no loss function or backpropagation, builds the fingerprint from scratch (s: 0.08 $\rightarrow$ 7.57) (Figure 2c).

Training on CIFAR-10 with randomly shuffled labels (100 epochs, 3 seeds) produces a stronger fingerprint than normal training (s=5.88 vs. 3.42), even though networks can only memorize ($\sim$10% test accuracy). The fingerprint tracks optimization dynamics, not task learning.

Model	Layers	Pair Acc	AUROC
GPT-2 (124M–1.5B)	12–48	**100%**	1.00
BERT-base / ViT-B/16	12	**100%**	0.97–1.00
Mistral-7B / LLaMA-2-7B	32	**100%**	0.96–1.00
LLaMA-2-7B-chat (+RLHF)	32	**100%**	0.96–1.00
Qwen2.5-7B / DeepSeek-R1	28–32	80–**100%**	0.93–1.00
Whisper (tiny/base/small)	4–12	**100%**	0.85–1.00
ResNet-50/101/152	5–35	91–100%	0.96–1.00

Table 3: Cross-architecture generalization. Pair accuracy (Section 4.1) via Hungarian matching on architecture-aware branch products. Ranges span model sizes within each family. Sub-100% SwiGLU sub-paths are recovered by joint factorization. Random-init baselines: $\leq$ 9%.

5.2 The Fingerprint Generalizes and Scales

We test whether the block alignment procedure of Section 4.1 recovers correct block correspondences across diverse architectures.

We extract architecture-aware branch products (Table 2) from 9 model families on Hugging Face (Hugging Face 2024; OpenAI 2024) (GPT-2 (Radford et al. 2019), BERT (Devlin

Pair (base vs.)	Kind	$\mathcal{L}$	W. cos	Verdict
Related (real fine-tunes)				
Llama-2-7B-chat	RLHF	$\mathbf{0.995}$	0.995	REL.
Vicuna-7B	instruct	$\mathbf{0.996}$	0.996	REL.
CodeLlama-7B	code PT	0.336	$\mathbf{0.414}$	REL.
Unrelated (cross-model null)				
OpenLLaMA-7B	scratch†	$\mathbf{4e{-}4}$	0.089	UNR.

Table 4: Public checkpoint lineage validation on the LLaMA-2 family (4 pairs). †OpenLLaMA shares LLaMA-2's exact architecture but is trained from scratch.

et al. 2019), LLaMA-2 (Touvron et al. 2023), Mistral (Jiang et al. 2023), Qwen (Yang et al. 2024), DeepSeek (DeepSeek-AI 2025) for language; ViT (Dosovitskiy et al. 2021), ResNet (He et al. 2016) for vision; Whisper (Radford et al. 2023) for speech) and apply Hungarian matching on the score matrix $s(i, j)$. Block-pairing accuracy reaches 80–100% across all 9 families, against random-initialization baselines of at most 9% (Table 3). The extraction transfers across GELU (Hendrycks and Gimpel 2016) and SwiGLU (Shazeer 2020) activations with no per-family tuning; only the factorization changes.

Extracting MLP branch products $M = W_{\text{proj}} W_{\text{fc}}$ from all four GPT-2 variants (hidden dim d=768/1024/1280/1600), we fit mean correct-pair score $\bar{s}(i, i)$ against dimension. The score grows as $d^{0.87}$ (4.18 → 5.46 → 6.98 → 7.77), reaching 15–19% of the theoretical $\sqrt{d}$ bound; score matrices are in Appendix C.2.

5.3 From Fingerprints to Lineage Verification

We evaluate the lineage score $\mathcal{L}$ on depth-24 MLPs (same architecture as Section 5.1). Related checkpoints are generated via fine-tuning, noise, pruning, and quantization; unrelated checkpoints come from independent training and distillation (159 pairs; see Reproducibility Statement). Checkpoints separate (Figure 1c). In Table 8, we observe fine-tune/quant/noise average $\mathcal{L}$=0.99 (min 0.97), pruning averages $\mathcal{L}$=0.81 (min 0.58 at 85% sparsity), while independent/distilled average $\mathcal{L}$=0.08. Extending to ResNet-18 on CIFAR-10, related checkpoints score $\mathcal{L}\geq$0.69 while independent models score $\mathcal{L}<$0.01 (AUROC=1.000; Appendix D.3).

On public LLM checkpoints (Table 4), we compare LLaMA-2-7B-base against 3 known derivatives (chat/RLHF, Vicuna/instruct-tuned, CodeLlama/code-pretrained) and 1 independently trained model (OpenLLaMA-7B), extracting SwiGLU branch products $M = W_{\text{down}} W_{\text{up}}$ across all 32 layers (hidden dim d=4096). Three fine-tuned variants score related ($\mathcal{L}$: 0.995, 0.996, 0.336); the independent scores at the 10^{-4} null. OpenLLaMA matches LLaMA-2 exactly (d=4096, L=32, d_{ff}=11008) yet was trained from scratch; its UNRELATED verdict can only come from weights. Weight cosine scores higher on CodeLlama, likely because continued pretraining on code shifts the signature more than simple fine-tuning. The score also decays with genealogical distance (Appendix B).

5.4 Lineage Verification Benchmark

We compare against six baselines on a 52-pair MLP benchmark (see Reproducibility Statement for construction details): three data-free weight-space methods (aligned Frobenius, singular-value distance, weight cosine) and three behavioral methods requiring 1024 probe samples each (SVCCA, CKA, and a regression analogue of IPGuard). Table 5 reports aggregate AUROC and Gap-$Z = (\bar{s}_{\text{related}} - \bar{s}_{\text{distilled}})/\sigma_{\text{distilled}}$, the standardized margin by which distilled students fall below related checkpoints. Four weight-space methods, ours included, reach AUROC=1.000 on this benchmark.

High AUROC on unmodified checkpoints could reflect true lineage detection or mere weight similarity. We disentangle them with *function-preserving laundering*: per-block permutation (P) and reciprocal rescaling (D) that leave the network's input–output map exactly unchanged while transforming the weights. Specifically, P permutes hidden units ($W_{\text{in}} \leftarrow PW_{\text{in}}$, $W_{\text{out}} \leftarrow W_{\text{out}}P^\top$), and D rescales them ($W_{\text{in}} \leftarrow DW_{\text{in}}$, $W_{\text{out}} \leftarrow W_{\text{out}}D^{-1}$). These transformations leave the branch product $M = W_{\text{out}}W_{\text{in}}$ exactly unchanged ($P^\top P = I$, $D^{-1}D = I$), so our signature is invariant by construction. Raw-weight baselines have no such guarantee.

Table 6 shows the results. Our method maintains AUROC=1.000 across all variants—the measured signature deviation is $|\Delta\mathcal{L}| \approx 10^{-8}$, confirming algebraic invariance. Raw-weight baselines collapse: aligned Frobenius drops to 0.495 under P alone and to 0.000 under any D; singular-value distance survives P (spectrum is permutation-invariant) but fails D; weight cosine partially survives both but degrades to 0.805 under PD.

We replicate the laundering experiment across four model families to confirm the invariance is not LLaMA-2-specific (Table 7). On each base→fine-tune pair, P permutes W_{gate}/W_{up} rows and W_{down} columns; D_{m} (mild rescaling, log-uniform $[0.5, 2]$) scales W_{up} rows against W_{down} columns. Our score is invariant: $\Delta\mathcal{L} < 10^{-7}$ across all families. Weight cosine collapses under P (0.95→0.003) but survives D_{m}. Unrelated controls (OpenLLaMA for LLaMA-2, Mistral for LLaMA-3) stay at the $\leq 10^{-4}$ null regardless of laundering, confirming no false positives.

On layer grafts and linear merges (Appendix D.1), $\mathcal{L}$ tracks the fraction of inherited blocks with Spearman $\rho \approx 0.96$, so the score measures partial overlap rather than merely crossing a threshold. Singular-value distance fails the merge regime (AUROC=0.448), and IPGuard inverts on grafts (AUROC=0.05), ranking higher-overlap suspects as *less* similar.

5.5 Survival Under Post-Training Modification

Section 5.3 established that the fingerprint detects lineage; here we measure how it degrades as transformation intensity grows. Table 8 covers five transformation families: fine-tuning (same and different target), Gaussian noise ($\sigma_{\text{rel}} \leq 15\%$), magnitude pruning (10–85%), quantization (16–256 levels), and LoRA adapter merging.

Fine-tuning, quantization, and noise leave the signal essentially intact (mean $\mathcal{L} = 0.99$, min 0.97), and LoRA-merged

Method	Type	Data-free	AUROC	Gap-Z ↑	Block Corr.
Diagonal Dominance (ours)	Weight-space	✓	**1.000**	$+53.0$	✓
Aligned Frobenius	Weight-space	✓	**1.000**	$\mathbf{+72.0}$	✓
Singular Value Distance	Weight-space	✓	**1.000**	$+7.9$	✗
Weight Cosine	Weight-space	✓	**1.000**	$\mathbf{+76.3}$	✗
SVCCA (Raghu et al. 2017)	Activation-space	✗	**1.000**	$+45.4$	✗
CKA (Kornblith et al. 2019)	Activation-space	✗	0.829	$+8.6$	✗
IPGuard (Cao, Jia, and Gong 2021) (regression analogue)	Decision-boundary	✗	0.700	-3.5	✗

Table 5: Lineage detection baselines (52-pair MLP). Data-free = no probe samples needed. Block Corr. = can identify which blocks match. Only our method and aligned Frobenius offer both.

Method	none	P	D	PD	PDFT
Ours (on M)	**1.000**	**1.000**	**1.000**	**1.000**	**1.000**
Raw Frobenius	**1.000**	0.495	0.000	0.000	0.000
Raw SVD dist	**1.000**	**1.000**	0.000	0.000	0.000
Raw weight cos	**1.000**	0.856	**1.000**	0.805	0.805

Table 6: AUROC under function-preserving laundering (52 pairs). P=permutation, D=rescaling (log-uniform $[0.1, 10]$), PDFT=P+D then 5 epochs fine-tuning. All variants pass a hard gate ($\max|\Delta f| < 10^{-4}$). Our score is invariant; raw-weight baselines collapse.

checkpoints score $\mathcal{L} \in [0.97, 0.99]$. Fine-tuning to a different target degrades the score to a mean of 0.94 (min 0.84). Pruning declines most steeply, from $\mathcal{L} = 0.99$ at 10% sparsity to 0.58 at 85%. The real-world extreme of the fine-tuning axis is CodeLlama: after heavy code-specialized continued pretraining it scores $\mathcal{L} = 0.336$, yet remains $\sim$700$\times$ above the unrelated null. Across all families, degradation correlates with utility loss ($\rho = -0.83$): no tested transformation removes the signal while preserving the model.

We also test adversarial suppression. A white-box adversary optimizes $\min_\Delta \mathcal{L}(A, A+\Delta) + \lambda \cdot \|f_{A+\Delta}(X) - f_A(X)\|^2$, sweeping $\lambda \in \{0, 10^{-2}, 10^{-1}, 1, 10\}$ (Appendix E). With $\lambda \geq 1$, the attack cannot push $\mathcal{L}$ below 0.91. Reaching the unrelated range ($\mathcal{L} \approx 0.08$) requires $\lambda = 10^{-2}$ and costs 12% eval-loss degradation; removing the utility constraint ($\lambda = 0$) destroys the model. The adversary faces the same coupling that makes benign transformations preserve the signal: the fingerprint lives in the correlated component of W_{in} and W_{out}, which encodes the branch's learned function.

6 Conclusion

A model's weights carry more history than they may appear to. We showed that training residual networks leaves a block-level trace in residual-branch products. By removing the diagonal component shared across trained models and comparing the remaining signatures, we can recover block correspondences and test whether two compatible checkpoints share weight ancestry. Across language, vision, and speech models, the signal remained detectable after fine-tuning, pruning, quantization, LoRA merging, and RLHF, while independently trained and distilled models remained distinct.

Base	Pair	Var	$\mathcal{L}$	W.cos
LLaMA-2 7B	chat	P	**.995**	.002
	chat	D_m	**.995**	.949
	OpenLLaMA†	P	5×10^{-5}	.000
	OpenLLaMA†	D_m	5×10^{-5}	.000
LLaMA-3 8B	Inst	P	**.995**	.003
	Inst	D_m	.995	**.954**
	Mistral†	P	0	.000
	Mistral†	D_m	0	.000
Mistral 7B	Inst	P	**.952**	.003
	Inst	D_m	.952	**.937**
Qwen2.5 7B	Inst	P	**.999**	.003
	Inst	D_m	.999	**.951**

Table 7: Function-preserving laundering on public checkpoints. P=permutation, D_m=mild rescaling (log-uniform $[0.5, 2]$). $\mathcal{L}$ is invariant ($\Delta\mathcal{L} < 10^{-7}$); weight cosine collapses under P but survives D_m. †Unrelated controls stay at $\leq 10^{-4}$ null.

These results establish residual-branch signatures as a practical post-hoc signal for weight-level lineage under a defined set of assumptions. The method requires white-box access and compatible residual architectures, and its symmetric score cannot determine the direction of descent. Within this scope, it provides a data-free and transformation-tolerant complement to hashes, metadata, and proactively inserted watermarks.

Limitations

Scope and architecture constraints. Our method needs white-box access to both checkpoints, so API-only models cannot be verified. It also only works for residual architectures: plain feedforward networks, RNNs, and state-space models are out of scope. Comparisons are further limited to models with matching depth and hidden dimension, so cross-architecture verification (e.g., LLaMA vs. GPT-2, or ViT vs. ResNet) is not supported.

Lineage determination. The lineage score is symmetric, so without external metadata we cannot tell which checkpoint is the ancestor. The score decays monotonically with genealogical distance, but the method compares single pairs and does not reconstruct multi-hop ancestry or family trees. Under partial inheritance (layer grafts, linear merges), the score reflects the fraction of shared blocks but not which blocks were inherited.

Signal degradation under transformation. The fingerprint survives common post-training modifications, but weakens as transformations grow more aggressive. Heavy pruning drops the signal to $\mathcal{L} = 0.58$ at 85% sparsity, and extensive continued pretraining (CodeLlama) lowers it to $\mathcal{L} = 0.336$, though both remain detectable above the null. A white-box adversary can push the score to chance, but only by sacrificing at least 12% utility; no tested attack removes the signal while preserving model function.

Calibration and validation. Verification requires calibrating a null distribution from independently trained models of the same architecture, and the threshold may need adjustment across architecture families. Our real-world validation covers a limited set of public checkpoints (four LLaMA-2 pairs plus four other model families), so broader ecosystem coverage is untested. SwiGLU architectures (Qwen2.5, DeepSeek-R1) also reach only 80% block-pairing accuracy, and need joint factorization to recover sub-paths reliably.

Reproducibility Statement

All results in this paper are produced by deterministic scripts with fixed seeds. The 52-pair MLP lineage benchmark (Tables 8, 5) uses depth-16 MLPs (in_dim=16, hidden=48) and is constructed from 2 trained reference checkpoints, each paired with 5 related-checkpoint kinds $\times 6$ derivative seeds $= 30$ truly-related pairs (fine-tune, fine-tune to a new target, $\sigma_{\text{rel}} \in [0.01, 0.15]$ Gaussian noise, 10–85% magnitude pruning, and 16–256-level uniform quantization), plus 16 different-seed same-task pairs and 6 distilled-student pairs as unrelated checkpoints. The expanded 159-pair set (Figure 1c) uses 3 depth-24 MLP reference models: each reference is paired with 25 related checkpoints (5 transformation types $\times$ 5 variations) and 28 unrelated checkpoints (15 independent same-task + 5 independent different-task + 5 random-init + 3 distilled), yielding $3 \times 25 = 75$ related pairs and $3 \times 28 = 84$ unrelated pairs ($75 + 84 = 159$ total). Exact parameters are listed in Appendix D; null calibration uses the 50 different-seed pairs from Appendix B. Real-architecture pairing (Table 3) loads public HuggingFace checkpoints. Code, configuration files, baseline implementations (aligned Frobenius, singular-value distance, weight cosine, CKA, SVCCA, IPGuard regression analogue), the per-pair score JSONs, and the plotting scripts will be released upon acceptance.

Ethics Statement

This work introduces a forensic method for identifying weight-level lineage between neural network checkpoints. Its intended purpose is to support legitimate provenance verification, e.g., helping model developers, platforms, and researchers determine whether one checkpoint was derived from another.

We recognize that the method has potential dual-use implications. In principle, it could be used to investigate the origins of models whose developers do not wish to disclose their lineage. However, the method requires white-box access to both the reference and suspect checkpoints, which substantially limits its use in adversarial settings. It also relies on a naturally occurring property of trained weights rather than an embedded watermark or tracking mechanism.

Overall, we believe the method's potential benefits—including intellectual property protection, license compliance, and greater transparency in model supply chains—outweigh these risks.

Use of AI Assistants

The authors bear full responsibility for all scientific claims, experimental results, and analysis in this work. During manuscript preparation, we used Claude to support literature searches, experimental design, code development, and the drafting and editing of text. All AI-assisted material was reviewed, verified, and revised by the authors.

References

Adi, Y.; Baum, C.; Cisse, M.; Pinkas, B.; and Keshet, J. 2018. Turning your weakness into a strength: Watermarking deep neural networks by backdooring. In *27th USENIX Security Symposium*, 1615–1631.

Ainsworth, S. K.; Hayase, J.; and Srinivasa, S. 2023. Git Re-Basin: Merging Models modulo Permutation Symmetries. In *International Conference on Learning Representations*.

Cao, X.; Jia, J.; and Gong, N. Z. 2021. IPGuard: Protecting Intellectual Property of Deep Neural Networks via Fingerprinting the Classification Boundary. In *AsiaCCS*, 14–25.

Dathathri, S.; See, A.; Ghaisas, S.; Huang, P.-S.; McAdam, R.; Welbl, J.; Bachani, V.; Kaskasoli, A.; Sherborne, T.; et al. 2024. Scalable Watermarking for Identifying Large Language Model Outputs. *Nature*.

DeepSeek-AI. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. *arXiv preprint arXiv:2501.12948*.

Dettmers, T.; Lewis, M.; Belkada, Y.; and Zettlemoyer, L. 2022. LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. In *Advances in Neural Information Processing Systems*.

Devlin, J.; Chang, M.-W.; Lee, K.; and Toutanova, K. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of NAACL-HLT*.

Dosovitskiy, A.; Beyer, L.; Kolesnikov, A.; Weissenborn, D.; Zhai, X.; Unterthiner, T.; Dehghani, M.; Minderer, M.; Heigold, G.; Gelly, S.; et al. 2021. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. In *Proceedings of ICLR*.

Han, S.; Mao, H.; and Dally, W. J. 2016. Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding. In *International Conference on Learning Representations*.

Han, S.; Pool, J.; Tran, J.; and Dally, W. J. 2015. Learning Both Weights and Connections for Efficient Neural Networks. In *Advances in Neural Information Processing Systems*.

He, K.; Zhang, X.; Ren, S.; and Sun, J. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 770–778.

Hendrycks, D.; and Gimpel, K. 2016. Gaussian Error Linear Units (GELUs). *arXiv preprint arXiv:1606.08415*.

Hinton, G.; Vinyals, O.; and Dean, J. 2015. Distilling the Knowledge in a Neural Network. *arXiv preprint arXiv:1503.02531*.

Horwitz, E.; Shul, A.; and Hoshen, Y. 2025. Unsupervised Model Tree Heritage Recovery. In *International Conference on Learning Representations*, 47900–47918.

Hugging Face. 2024. Hugging Face Model Hub. https://huggingface.co/models. Accessed: 2024.

Ilharco, G.; Ribeiro, M. T.; Wortsman, M.; Gururangan, S.; Schmidt, L.; Hajishirzi, H.; and Farhadi, A. 2023. Editing Models with Task Arithmetic. In *International Conference on Learning Representations*.

Jia, H.; Yaghini, M.; Choquette-Choo, C. A.; Dullerud, N.; Thudi, A.; Chandrasekaran, V.; and Papernot, N. 2021. Proof-of-Learning: Definitions and Practice. In *IEEE Symposium on Security and Privacy*, 1039–1056.

Jiang, A. Q.; Sablayrolles, A.; Mensch, A.; Bamford, C.; Chaplot, D. S.; de las Casas, D.; Bressand, F.; Lengyel, G.; Lample, G.; Saulnier, L.; et al. 2023. Mistral 7B. *arXiv preprint arXiv:2310.06825*.

Kornblith, S.; Norouzi, M.; Lee, H.; and Hinton, G. 2019. Similarity of Neural Network Representations Revisited. In *International Conference on Machine Learning*, 3519–3529.

Krizhevsky, A. 2009. Learning Multiple Layers of Features from Tiny Images. Technical report, University of Toronto.

Kuhn, H. W. 1955. The Hungarian Method for the Assignment Problem. *Naval Research Logistics Quarterly*, 2(1-2): 83–97.

Le Merrer, E.; Perez, P.; and Trédan, G. 2020. Adversarial Frontier Stitching for Remote Neural Network Watermarking. *Neural Computing and Applications*, 32(13): 9233–9244.

Lukas, N.; Jiang, E.; Li, X.; and Kerschbaum, F. 2022. SoK: How robust is image classification deep neural network watermarking? In *2022 IEEE Symposium on Security and Privacy (SP)*, 787–804. IEEE.

Mitchell, M.; Wu, S.; Zaldivar, A.; Barnes, P.; Vasserman, L.; Hutchinson, B.; Spitzer, E.; Raji, I. D.; and Gebru, T. 2019. Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency*, 220–229.

OpenAI. 2024. GPT-2 Models on Hugging Face. https://huggingface.co/openai-community. Accessed: 2024.

Ouyang, L.; Wu, J.; Jiang, X.; Almeida, D.; Wainwright, C. L.; Mishkin, P.; Zhang, C.; Agarwal, S.; Slama, K.; Ray, A.; et al. 2022. Training Language Models to Follow Instructions with Human Feedback. In *Advances in Neural Information Processing Systems*.

Park, H. 2026. I Dropped a Neural Net. arXiv:2602.19845.

Pennington, J.; Schoenholz, S.; and Ganguli, S. 2017. Resurrecting the sigmoid in deep learning through dynamical isometry: theory and practice. In *Advances in Neural Information Processing Systems*, volume 30.

Radford, A.; Kim, J. W.; Xu, T.; Brockman, G.; McLeavey, C.; and Sutskever, I. 2023. Robust Speech Recognition via Large-Scale Weak Supervision. In *Proceedings of ICML*.

Radford, A.; Wu, J.; Child, R.; Luan, D.; Amodei, D.; and Sutskever, I. 2019. Language Models are Unsupervised Multitask Learners. *OpenAI Technical Report*.

Raghu, M.; Gilmer, J.; Yosinski, J.; and Sohl-Dickstein, J. 2017. SVCCA: Singular Vector Canonical Correlation Analysis for Deep Learning Dynamics and Interpretability. In *Advances in Neural Information Processing Systems*.

Shazeer, N. 2020. GLU Variants Improve Transformer. *arXiv preprint arXiv:2002.05202*.

Touvron, H.; Lavril, T.; Izacard, G.; Martinet, X.; Lachaux, M.-A.; Lacroix, T.; Rozière, B.; Goyal, N.; Hambro, E.;

Azhar, F.; et al. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.

Uchida, Y.; Nagai, Y.; Sakazawa, S.; and Satoh, S. 2017. Embedding watermarks into deep neural networks. In *Proceedings of the 2017 ACM on International Conference on Multimedia Retrieval*, 269–277.

Wortsman, M.; Ilharco, G.; Gadre, S. Y.; Rober, R.; Morcos, A.; Hajishirzi, H.; Farhadi, A.; Kornblith, S.; and Schmidt, L. 2022. Model Soups: Averaging Weights of Multiple Fine-tuned Models Improves Accuracy without Increasing Inference Time. In *International Conference on Machine Learning*, 23965–23998.

Yang, A.; Yang, B.; Zhang, B.; Hui, B.; Zheng, B.; Yu, B.; Li, C.; Liu, D.; Huang, F.; Wei, H.; et al. 2024. Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115*.

Zeng, B.; Wang, L.; Hu, Y.; Xu, Y.; Zhou, C.; Wang, X.; Yu, Y.; and Lin, Z. 2025. HuRef: HUman-REadable Fingerprint for Large Language Models. arXiv:2312.04828.

Zhang, J.; Liu, D.; Qian, C.; Zhang, L.; Liu, Y.; Qiao, Y.; and Shao, J. 2025. REEF: Representation Encoding Fingerprints for Large Language Models. In *International Conference on Learning Representations*, 48092–48117.

A Extended Theoretical Results

A.1 Null Model (Corollary)

Corollary 2 (Null Model)**.** *For randomly initialized weights with no training, all pairs satisfy* $\mathbb{E}[s(i,j)] = 1/\sqrt{d} + o(1)$ *uniformly in* (i,j)*. Hungarian matching recovers correct pairs at the chance rate* $1/L$*.*

The null model rules out three alternative explanations:

1. Diagonal dominance is not an artifact of compatible matrix shapes
2. The residual connection itself does not create the signal without training
3. Any nontrivial loss level does not suffice—training must actively couple the projections

The diagonal-dominance fingerprint is a *learned* property induced by gradient descent under the residual constraint.

B Verification Protocol Details

B.1 Gated Branch Score

Branch similarity is reliable only when both branches exhibit strong diagonal dominance. We gate the cosine similarity:

$$G(\phi_\ell^A, \phi_m^B) = \langle \phi_\ell^A, \phi_m^B \rangle \cdot \min\Big(\tfrac{s(M_\ell^A)}{\tau_s}, \tfrac{s(M_m^B)}{\tau_s}, 1\Big) \quad (10)$$

where τ_s is the minimum diagonal-dominance ratio across reference branches.

B.2 Statistical Calibration

We calibrate against a null distribution $\mathcal{N}_A = \{\mathcal{L}(A, U_j) : U_j \notin \mathcal{D}(A)\}$ of scores between reference A and independently trained models. The z-score is:

$$Z(A,B) = \frac{\mathcal{L}(A,B) - \mu_{\text{null}}}{\sigma_{\text{null}}} \quad (11)$$

Under Gaussian approximation, $\tau = 1.645$ yields 95% confidence; $\tau = 3.0$ yields FPR $< 0.13\%$ for high-stakes applications.

Transformation	Mean $\mathcal{L}$	Min $\mathcal{L}$	Det. @ 1%FPR
Fine-tune / Quant / Noise	0.99	0.97	45/45
Fine-tune (diff. target)	0.94	0.84	15/15
Pruning (10–85%)	0.81	0.58	15/15
Independent / Distilled	0.08	0.01	-

Table 8: Lineage score survival under post-training modification (n=75 related, n=84 unrelated). All related checkpoints detected at 1% FPR; unrelated stay below 0.20.

C Mechanism Validation

C.1 Initialization Ablation

Table 9 shows pair accuracy before and after training across seven initialization schemes on depth-24 residual MLPs (3 seeds, 200 epochs). All schemes show chance-level accuracy ($\leq$2.1%) at initialization, rising to 93–100% after training. Orthogonal initialization provides dynamical isometry at init yet shows zero correct pairs before training.

Init scheme	Untrained	Trained	AUROC
Orthogonal	0.0%	100%	0.947
Kaiming-normal	2.1%	97%	0.981
Kaiming-uniform	2.1%	98.6%	0.98
Xavier-normal	2.1%	100%	0.990
Xavier-uniform	2.1%	100%	0.99
Uniform	2.1%	93.8%	—
Gaussian σ=0.02	2.1%	13%	0.671

Table 9: Initialization ablation on depth-24 residual MLPs. All schemes show chance-level accuracy before training. The Gaussian σ=0.02 case fails because blocks collapse to near-zero contribution.

C.2 GPT-2 Scaling, Trace, and Jacobian Analysis

Table 10 reports the normalized Jacobian deviation $\delta_J^{\text{norm}} = \|J^\top J/\|J\|_F^2 - I/d\|_F$ across GPT-2 scales. This metric is scale-invariant: any scaled orthogonal matrix scores 0, and larger values indicate non-uniform singular values.

Figure 3 shows block pairing score matrices $s(i,j)$ for GPT-2 models from 124M to 1.5B parameters.

Figure 4 shows the trace values $\text{tr}(W_{\text{out}}W_{\text{in}})$ per block across all four GPT-2 variants.

C.3 Alternative Methods Comparison

Figure 5 compares block pairing accuracy across GPT-2 scales.

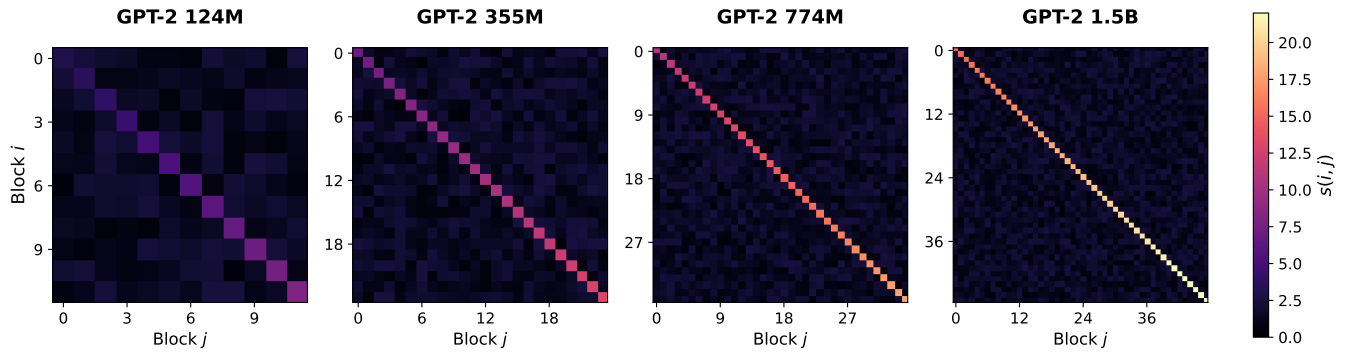


Figure 3: Block pairing score matrices $s(i, j)$ for GPT-2 models from 124M to 1.5B parameters. Diagonal entries dominate, yielding 100% pairing accuracy across all scales.

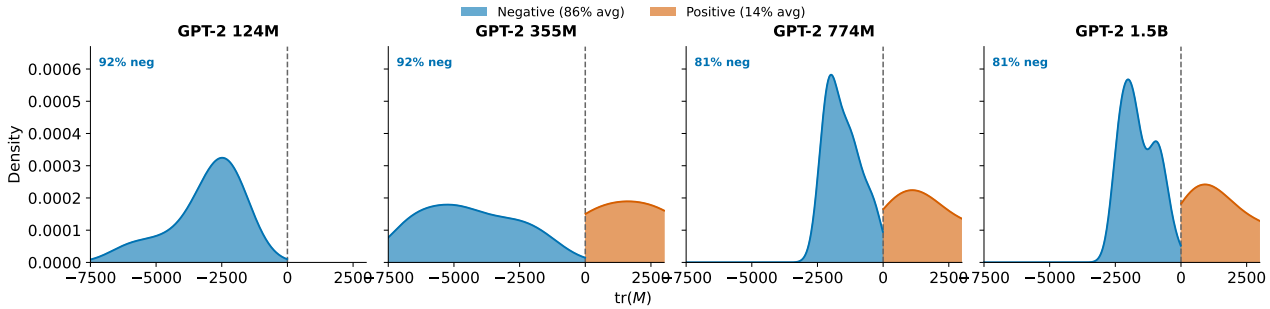


Figure 4: Distribution of trace values $\mathrm{tr}(W_{\text{out}}W_{\text{in}})$ across GPT-2 scales. Blue: negative trace (86% average); orange: positive trace (14% average). The strong skew toward negative values confirms the diagonal-dominance mechanistic prediction.

Model	d	Pretrained δ_J^{norm}	Random-init δ_J^{norm}	Ratio
GPT-2-small	768	0.297	0.025	11.7×
GPT-2-medium	1024	0.242	0.026	9.4×
GPT-2-large	1280	0.155	0.025	6.2×
GPT-2-XL	1600	0.122	0.024	5.1×

Table 10: Jacobian orthogonality across GPT-2 scales. Pretrained models are 5–12× less orthogonal than at random initialization, falsifying the dynamical isometry hypothesis. Random-init δ_J^{norm} is nearly constant across scales ($\approx$0.025); pretrained values decrease with model size but never approach the random-init floor.

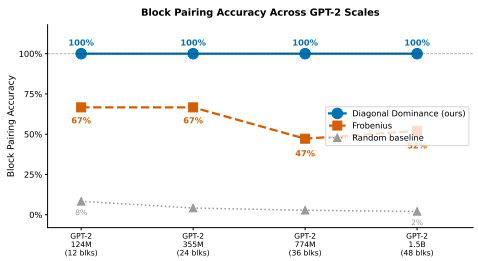


Figure 5: Block pairing accuracy as model size increases. Diagonal dominance (blue) maintains 100% accuracy; Frobenius matching (orange) degrades from 67% to 52%; random baseline (gray) drops from 8% to 2% as the number of blocks increases.

D Extended Benchmarks

D.1 Harder Regime: Layer Grafts and Linear Merges

We construct 40 harder pairs from depth-16 MLPs (same setup as Table 5): (i) **layer grafts** copy $K \in \{0, 4, 8, 12, 16\}$ of 16 blocks from reference into an independent donor (related if $K \geq 8$); (ii) **linear merges** compute $w \cdot \text{ref} + (1 - w) \cdot \text{donor}$ with $w \in \{0, 0.25, 0.5, 0.75, 1.0\}$ (related if $w \geq 0.5$). We use 2 references and 2 donors per reference. AUROC measures binary related-vs-unrelated detection; Spearman correlation measures whether a method tracks partial overlap monotonically (K/L or w) rather than just clearing a threshold. Table 11 summarizes the 40-pair benchmark.

D.2 ResNet Factorization

On ImageNet-pretrained ResNet-50/101/152 (torchvision), we compare naive two-layer factorization (W_3W_1, skip-

Method	AUROC		Spearman ρ	
	Graft	Merge	Graft	Merge
Diagonal dominance (ours)	0.979	1.000	+0.96	+0.96
Aligned Frobenius	1.000	1.000	+0.99	+0.99
Weight cosine	1.000	1.000	+0.98	+0.96
Singular-value distance	1.000	0.448	+0.99	+0.23
SVCCA	1.000	1.000	+0.97	+0.99
CKA	0.813	0.781	+0.73	+0.75
IPGuard (regr. analogue)	0.052	0.448	−0.83	+0.22

Table 11: Harder-regime benchmark on layer grafts and linear merges (40 pairs). Related: $K \geq L/2$ for grafts, $w \geq 0.5$ for merges. Weight-space methods (ours, aligned Frobenius, weight cosine) and SVCCA maintain AUROC≈1.000 with $\rho \geq 0.96$. CKA degrades to AUROC≈0.80; IPGuard collapses on grafts (AUROC=0.05, negative ρ); singular-value distance fails merges ($\rho = 0.23$). Our method tracks partial overlap monotonically ($\rho \geq 0.96$), not just binary detection.

ping the middle conv) against the correct three-layer product ($W_3W_2W_1$) on layer3 bottleneck blocks (n=5/22/35 blocks respectively). Naive W_3W_1 achieves chance-level accuracy; the correct triple product recovers:

- ResNet-50/layer3 (n=5): 100%, AUC 1.000
- ResNet-101/layer3 (n=22): 100%, AUC 0.995
- ResNet-152/layer3 (n=35): 91%, AUC 0.964

D.3 CIFAR-10 ResNet-18 Benchmark

We train two CIFAR-10 ResNet-18 references and three independents. Each reference yields 11 related checkpoints (noise $1-10\%$, pruning $20-80\%$, quantization $16-256$ levels). Results: AUROC=1.000, TPR@1%FPR=100%. Related checkpoint scores: $\mathcal{L} \in [0.695, 1.000]$; unrelated baseline: $\mathcal{L}_{\max} = 0.004$.

D.4 ROC for Lineage Verification

Figure 6 shows that diagonal dominance alone cannot distinguish related from unrelated trained models.

D.5 Branching Ancestry Recovery

From the MLP benchmark (Table 8 setup), we construct a family tree: root C_1 (trained from scratch), siblings C_2 and C_3 (both fine-tuned from C_1), grandchildren C_4 (from C_2) and C_5 (from C_3). We then measure $\mathcal{L}(C_4, \cdot)$ against all candidates. The lineage score ranks candidates correctly:

$$\begin{aligned}\mathcal{L}(C_4, C_2) = 0.961 \;&>\; \mathcal{L}(C_4, C_1) = 0.910 \\ &>\; \mathcal{L}(C_4, C_3) = 0.867 \;>\; \mathcal{L}(C_4, C_5) = 0.850 \\ &\gg\; \mathcal{L}(C_4, I_k) \approx 0.08.\end{aligned}$$

The score decays monotonically with genealogical distance while maintaining ≈10× family/strangers separation.

E Attack and Robustness Details

E.1 Gradient-Based Suppression Attack

An adversary optimizes perturbation Δ to minimize lineage score subject to utility:

$$\min_{\Delta} \; \mathcal{L}_{\cos}(A, A + \Delta) \;+\; \lambda \cdot \|f_{A+\Delta}(X) - f_A(X)\|_2^2 \quad (12)$$

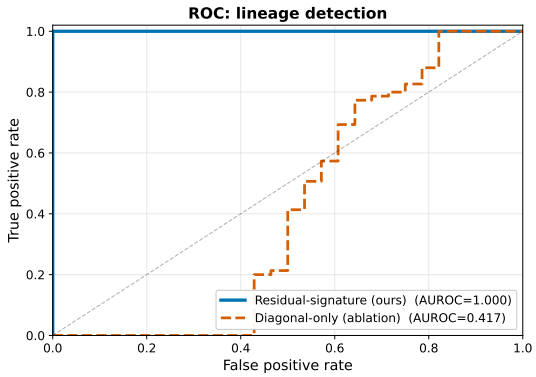


Figure 6: ROC for lineage verification on depth-24 residual MLPs. The residual-signature score achieves AUROC=1.000; diagonal-dominance alone fails (AUROC=0.417) because it cannot distinguish two trained residual models.

λ	Final $\mathcal{L}$	Eval loss	Verdict
0	0.053	39.5	utility destroyed
10^{-2}	0.054	0.77	+12% loss
10^{-1}	0.083	0.70	+1.5% loss
≥ 1	0.91+	0.69	utility preserved

Table 12: Pareto frontier: suppressing $\mathcal{L}$ to chance level requires ≥12% utility loss.